



MEC Defense: Securing Smart Stadium Critical Infrastructure

Universidade de Aveiro – Instituto de Telecomunicações

Orientadores:

Professor Daniel Corujo

Professor Vitor Cunha

José Fernandes nº 114472

Diogo Ferreira nº 114002

Rúben Franco nº 114827

Diogo Carvalho nº 113221

Gonçalo Martins nº 112678

Tiago Rocha nº 120515

Abstract

Modern high-density venues are evolving into massive smart environments, turning regular networks into a shared environment composed of local services, smart sensors and cameras, traffic from regular visitors, critical services, and much more.

Multi-Access Edge Computing (MEC) centres data processing on a local server and creates an overall faster network with lower latency and more privacy. Although the processing power being closer to the user can make it faster, the security concerns also increase, given that the worst threats are usually from inside the network, and this architecture creates a high dependency on internal processing. All this creates the need for a security framework. The purpose of this project is to fill that need by providing logs from all across the MEC system to the platform owner or Security Operations Center (SOC) analyst.

The final result should identify and alert different attacks and provide a good log set for posterior analysis. It is important to note that this would be only a part of a larger security plan including more mitigation methods and preventive measures.

Contents

1	Introduction	1
2	State of the art	2
2.1	Related work	2
2.1.1	Automatic Service Migration in a MEC-NFV Environment	2
2.1.2	MEC in OSM	3
2.2	Technologies	4
3	Conceptual Modeling	6
3.1	Requirements	6
3.2	Personas	6
3.3	Use cases	7
3.3.1	Real-Time Security Monitoring	7
3.3.2	SSH Brute Force Detection and Mitigation	7
3.3.3	Forensic Analysis	8
4	Architecture	9
4.1	Main System	9
4.2	Log Collection	11
4.2.1	Suricata, Zeek and the App	11
4.2.2	Prometheus	11

4.2.3	Falco	11
4.3	MEC Demo App	12
4.4	Network	12
4.5	Rules	13
4.6	Whitelisted IPs	14
4.7	Metrics-based Alerts	15
4.8	Log History	15
5	Results	16
5.1	Security Information and Event Management SIEM Integration	16
5.2	Monitoring and Telemetry	17
5.3	Threat Detection and Automated Response	19
5.4	Network Isolation and Traffic Segmentation	20
6	Conclusion	22
6.1	Future work	22

List of Figures

2.1	The proof of concept architecture as presented in figure 5 of the paper.	2
2.2	Example app descriptor	4
2.3	Customer Facing Service (CFS) portal dashboard listing the available applications and created instances	4
4.1	Deployment diagram of the Security Information and Event Management (SIEM) and the monitored MEC application pods	10
4.2	Main architecture of the project	10
4.3	Minimum required app logs	12
4.4	Small segment of the deployment	12
4.5	App demonstrative rules	13
4.6	Rule that identifies the expired certificate tag in the log	14
4.7	Rule that identify port scanning	14
4.8	Format of whitelisted IPs	14
4.9	Rule that alerts the user of a non-whitelisted connection	14
4.10	List of hardware alert rules inside Prometheus	15
5.1	Wazuh Dashboard displaying alert severity breakdown and available security modules	16
5.2	Three deployed and active agents registered in the MEC deployment	17
5.3	Dashboard showing the video processing application agent, with Suricata and application log events	17
5.4	Grafana dashboard showing per-node CPU, memory, and network telemetry collected via Prometheus Node Exporter.	18

5.5	Grafana Kubernetes cluster dashboard displaying aggregate resource usage across all nodes and pods.	18
5.6	Example of a Discord notification dispatched by Alertmanager upon detection of high CPU usage (top) and the corresponding resolution notification once the metric returned to normal (bottom)	19
5.7	Example of a Discord notification triggered by a Zeek alert for a connection originating from a non-whitelisted IP address.	19
5.8	Simulated SSH brute-force attack from an external host (left), UFW block logs at the firewall (top right), and the automated IP ban webhook log confirming mitigation (bottom right).	20
5.9	Logs from the firewall	20
5.10	Empirical verification of VXLAN-based network isolation: cross-segment ping attempts result in 100% packet loss, with UFW block entries confirming firewall enforcement.	21
5.11	Throughput measurements using <code>iperf3</code> within the MEC internal segment (left, ~4 Gbits/s) and the public segment (right, ~1.91 Gbits/s), confirming independent operation of both networks.	21

Abbreviations

AES Advanced Encryption Standard

CFS Customer Facing Service

ETSI European Telecommunications Standards Institute

IDS Intrusion Detection System

IPS Intrusion Prevention System

k8s Kubernetes

LCM Life Cycle Management

MEAO Mobile Edge Application Orchestrator

MEC Multi-Access Edge Computing

MANO Management and Orchestration

NFV Network Function Virtualization

NBI Northbound Interface

NG-UI Next Generation User Interface

NG-SA Next Generation Service Assurance

NFVI Network Functions Virtualization Infrastructure

OSM Open Source Management and Orchestration (MANO)

OSS Operating Support System

RO Resource Orchestrator

SIEM Security Information and Event Management

SLA Service-level Agreement

SOC Security Operations Center

uRPF Unicast Reverse Path Forwarding

VNF Virtualized Network Function

XDR Extended Detection and Response

Acknowledgement of Use of AI Tools

The authors acknowledge the use of AI-assisted tools, during the development of this project. These tools were used to support tasks such as writing assistance and code generation. All outputs were reviewed, validated, and adapted by the authors, who retain full responsibility for the content of this report.

Chapter 1

Introduction

The ever-increasing network demands of modern high density venues have led to the emergence of what are now called "Smart Environments". To meet these demands, the industry has moved away from the traditional cloud model and towards bringing compute closer to the user. As a result, the infrastructure must now support the visitor traffic as well as the local services, sensors, and cameras on the same network.

Multi-Access Edge Computing (MEC), standardized by the European Telecommunications Standards Institute (ETSI), is the most prominent architecture to surface from this shift. By moving compute away from a centralized server to the edge of the network, MEC delivers the performance and control these environments require. In doing so, however, it concentrates critical processing inside the local network where the attack surface is significantly harder to control.

While ETSI's standards provide a baseline security posture for MEC deployments and do acknowledge the importance of log collection and security monitoring, a complete and unified framework for implementing operational visibility does not currently exist. Without it, platform owners and SOC analysts may be limited in their ability to detect anomalous behavior, trace incidents, or respond to attacks and malfunctions in real time.

This project addresses that gap by designing and implementing a log and metrics-based security monitoring framework for MEC environments. By collecting and centralizing logs and metrics from across the stack, the system provides operators with meaningful visibility into the network, generating alerts on suspicious activity and automating defensive responses where applicable. It does not stand as a standalone security solution, but rather as a single component of a broader security plan.

This report begins with a review of the state of the art and relevant technologies, followed by the conceptual model, the proposed architecture, and finally the results and conclusions drawn from the implementation.

Chapter 2

State of the art

2.1 Related work

Our work is built on top of the MEC infrastructure developed by Pedro Pereira, Pedro Escaleira, Diogo Gomes, and Rui Aguiar. This architecture established the foundations on which this project operates. While their work focused on service reliability and scalability, namely ensuring MEC applications remain operational through automated migration, ours focuses on security, hardening that same infrastructure through a layered monitoring and detection stack covering log aggregation, real-time threat detection, and automated defensive responses.

2.1.1 Automatic Service Migration in a MEC-NFV Environment

The paper Automatic Service Migration in a MEC-Network Function Virtualization (NFV) Environment [3] presents an architecture for a MEC-NFV network that allows the integration of MEC with NFV orchestration through ETSI's Open Source MANO (OSM) for automated implementation and management of edge services. The paper proposes automatic service migration based on previously agreed Service-level Agreements (SLAs), controlled by Mobile Edge Application Orchestrator (MEAO), delegating security to the underlying technologies.

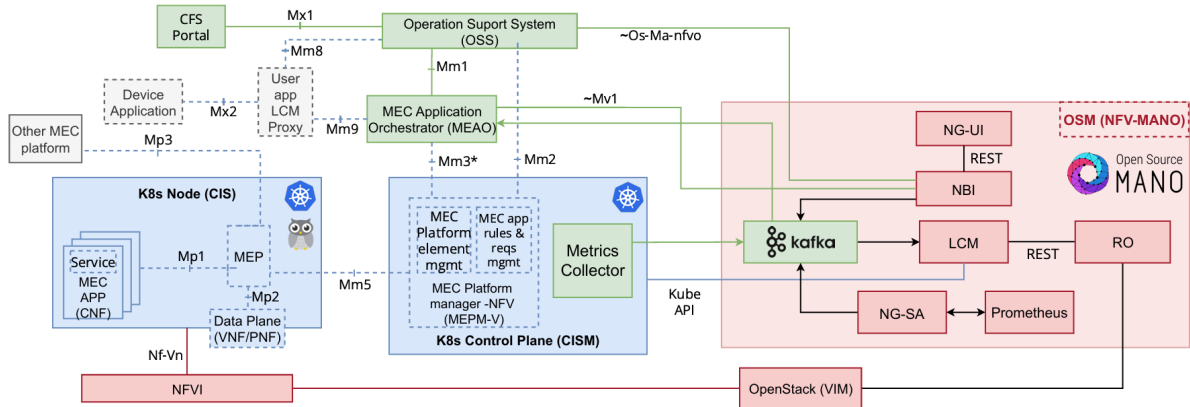


Figure 2.1: The proof of concept architecture as presented in figure 5 of the paper.

The architecture shown in the paper contains a normal OSM installation, that includes the following red blocks:

- Next Generation User Interface (NG-UI)
Used to allow communication to external systems (like MEAO) to send commands to OSM;
- Northbound Interface (NBI)
The NBI is a REST API client that allows end-users to manage the entire ecosystem and corresponding VNFs;
- Life Cycle Management (LCM)
The LCM is responsible for managing and orchestrating tasks;
- Resource Orchestrator (RO)
Takes the deployment requests from LCM and translates them into specific resource allocations;
- Next Generation Service Assurance (NG-SA)
Used to monitor the health, metrics, and alarms of the deployed network functions;
- Kafka
A distributed event streaming platform used as the internal message bus for OSM components to communicate with each other;
- OpenStack(VIM)
The software that controls and provisions the virtual machines the resources Kubernetes needs.
- Network Functions Virtualization Infrastructure (NFVI)
The physical hardware on each the infrastructure runs. Usually at the edge layer.

The OSM is configured to connect to a Kubernetes (k8s) cluster identified by the blue blocks. This will be the main focus point of this project, and where most of the logs come from. In this section the main blocks are the Service/MEC Apps, which are the ones being monitored, the Data Plane (Virtual/Physical Network Function), which manages the routing, and switching the user data traffic to the correct MEC app, and the Metrics Collector, which gets the cluster metrics, and through cAdvisor gathers each MEC App metrics.

Between each block it is possible to identify the interface it operates in.

2.1.2 MEC in OSM

The MEC in OSM project [2] implements a MEC network on top of an existing ETSI OSM installation and a Kubernetes cluster. It provides a MEAO service, an Operating Support System (OSS) to redirect requests to the service, and the CFS portal shown in figure 2.3 for administrators to upload and instantiate MEC applications as Virtualized Network Functions (VNFs).

MEC applications are standard containerized applications packaged for Kubernetes with Helm, and are accompanied by an app descriptor (which maybe written from an SLA) describing the required conditions and resources such as memory and processing power. The descriptor's fields are defined by ETSI [1]. An example of such a descriptor is given in listing 2.2.

```

mec-appd:
  id: mec-test-server-appd
  name: mec-test-server-appd
  provider: IT
  soft-version: "1.0"
  version: '1.0'
  mec-version: '1.0'
  info-name: Test Server MEC App
  description: Server to test the
    MEC migration and
    deployment process
  ext-cpd:
    - id: mgmtnet-ext
      k8s-cluster-net: mgmtnet
      layer-protocol:
        - "IPv4"
  artifacts:
    - name: mec-test-server
      description: Helm Chart for Ditto
      helm-chart: mec-test-server
      file: helm.tar.gz
      type: helm3
  migration-policy:
    - artifact: mec-test-server
      enabled: 1
      cpu-criteria:
        allocated-cpu: 1
        cpu-surge-capacity: 1
        cpu-threshold-time: 0
        cooldown-time: 0
      mem-criteria:
        allocated-mem: 1024
        mem-surge-capacity: 0
        mem-threshold-time: 0
        cooldown-time: 0
      mobility-criteria:
        mobility-migration-factor: 0
        mobility-threshold-time: 0
        cooldown-time: 0

```

Figure 2.2: Example app descriptor

2.2 Technologies

The technological stack needed for the development of this project includes the following tools.

- **SIEM / Threat Intelligence: Wazuh**

Wazuh is an open-source cybersecurity platform, unifying Extended Detection and Response (XDR) and SIEM in a single solution. It analyzes security data across endpoints, clouds, and networks to detect threats, respond to incidents, and ensure compliance. It is mainly used as highly compatible and configurable central monitoring and log analysis system. Wazuh is installed with various modules which support actions like Threat Intelligence with MITRE ATT&CK, where the user can get details about the threat actors, attack tactics, and techniques used in a security event;

- **Network Traffic Analysis and Intrusion Detection System (IDS)/Instruction Prevention System (IPS): Suricata and Zeek**

Suricata is a high-performance IDS and IPS that generates logs in case of anomalies present in the network;

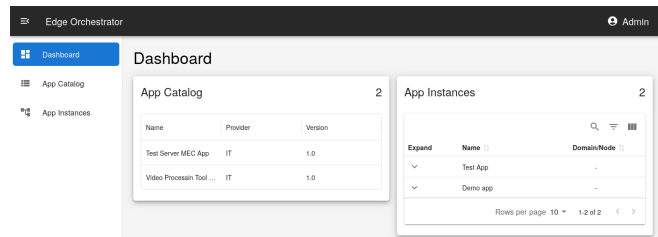


Figure 2.3: CFS portal dashboard listing the available applications and created instances

Much like Suricata, Zeek analyzes network activity but its focus is to provide context logs to possible intrusions or attacks. It is highly valuable for forensic analysis and cross-referencing events;

- **Packet Monitoring and Container Security:** Falco

Falco is a cloud native security tool that provides runtime security across hosts, containers, Kubernetes, and cloud environments. It leverages custom rules on Linux kernel events and other data sources through plugins, enriching event data with contextual metadata to deliver real-time alerts;

- **Infrastructure:** Kubernetes and Linux Namespaces/cgroups

Kubernetes, powered by Linux Namespaces and cgroups, is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications;

- **Log Normalization:** Filebeat

Filebeat is a lightweight, open-source log-shipping agent used to centralize and standardize logs before being sent to the SIEM;

- **Telemetry and Metrics Visualization:** Prometheus and Grafana

Prometheus is an open-source time-series metrics and monitoring suite that can be configured to, through alertmanager, alert the SysAdmin or other services in case of anomalous behavior. Data can be directly queried or visualized in real-time from within Grafana.

Grafana is an open-source web application dedicated to interactive visualization and analytics. It serves as one of the primary visual interfaces for the SOC analyst, querying real-time telemetry from Prometheus and presenting it as intuitive and easily readable dashboards, ensuring real-time operational visibility.

- **Protocols and Automation:** UFW (iptables), VXLANs, Python.

Packet filtering, port forwarding and NAT are leveraged by iptables and UFW, technologies that utilize the Linux kernel's Netfilter framework under-the-hood.

VXLAN is an encapsulation protocol, that takes Layer 2 traffic and encapsulates it in Layer 3 traffic (UDP) allowing it to cross routers. It uses a 24-bit ID, supporting over 16 million networks.

Python is a high-level, interpreted, general-purpose programming language that emphasizes code readability and simplicity that we used for alert management and firewall automation.

Chapter 3

Conceptual Modeling

3.1 Requirements

This section presents the specification of the system requirements, which were obtained based on the problem analysis, the study of the state of the art, and the needs identified through the defined personas. The requirement elicitation process took into account the dual nature of the system under development. On one hand, the need to provide robust security mechanisms capable of protecting critical MEC infrastructure from both internal and external threats. On the other hand, the need to ensure that such mechanisms do not interfere with the quality of service experienced by end users, namely stadium guests accessing the public network.

Functional Requirements	Non-Functional Requirements
- Real-time log aggregation and visualization via a SOC dashboard - Custom rules and filters - Identifying threats within the network - Isolation of each application and traffic segmentation - Correlation of logs and automated response systems	- Low latency - Scalability - Reliability - High Performance - Auditability and Resilience

3.2 Personas

The personas were defined with the goal of representing the main user profiles of the system, allowing for the identification of specific needs, objectives, and challenges. This approach helps guide the development of the solution, ensuring alignment with functional requirements and the context of use.

Name: Chuck Keith Age: 35 Profession: SOC Analyst	Wants to monitor real-time high importance traffic in the MEC system to prevent and assure security to Apps and Services present within, while not needing to analyse hundreds of dispensable logs.
--	---

Table 3.1: Persona 1

Name: João Malhoa Age: 50 Profession: Network Engineer	Rents a MEC system for large events (such as football games) and wants a quick implementation of the framework to ensure network monitoring.
---	--

Table 3.2: Persona 2

3.3 Use cases

3.3.1 Real-Time Security Monitoring

Actor: Chuck Keith (SOC Analyst)

Description: During an event at a venue, Chuck monitors the MEC system in real-time through the Wazuh dashboard, observing overall system resources, important system events, and active alerts generated by Suricata and Zeek.

Preconditions: The security framework has been deployed and is active during the event.

Main Flow:

1. The analyst opens the Wazuh dashboard.
2. Suricata, Zeek and Prometheus report network anomalies, which are automatically displayed as alerts in the dashboard.
3. The analyst inspects the logs and evaluates the severity of a flagged event.
4. The analyst can now decide whether or not to take action.

Postconditions: The threat is preserved in a log history and the response is triggered.

3.3.2 SSH Brute Force Detection and Mitigation

Actor: SIEM (automated)

Description: The SIEM automatically detects a brute force ssh entry originating from the public network and applies mitigation rules.

Preconditions: Custom detection rules are configured in the SIEM and the response scripts are active.

Main Flow:

1. Zeek identifies an abnormal traffic pattern that could indicate a SSH brute force attack targeting the NFVI.
2. An alert is sent out to Wazuh, which triggers a pre-configured response.
3. The offending traffic is redirected to an isolated cloud environment to facilitate forensic analysis, apply defensive responses and keep the system safe.
4. The SOC analyst is notified and the incident is logged.

Postconditions: The attack is redirected and the system is unaffected.

3.3.3 Forensic Analysis

Actor: João Malhoa (Network Engineer)

Description: After the event has ended, the network engineer grabs the logs from the SIEM to analyse and generate a security report.

Preconditions: The event has ended and the logs were captured and archived successfully.

Main Flow:

1. The engineer goes to Wazuh and requests the log archive.
2. The logs are congregated and cross-referenced to create a timeline of events.
3. The engineer can then summarise the event and generate a security report.

Postconditions: The logs and report are archived for compliance purposes.

Chapter 4

Architecture

4.1 Main System

This project is based on an already existing proof of concept network architecture presented in figure 2.1, with the MEC in OSM project installed on top for deployment of MEC applications.

The main development area of this project is, as previously mentioned, located in the k8s' nodes represented by the blue blocks in the figure 2.1. The applications are deployed on MEC and are packaged with Helm, which allows for many types of Kubernetes resources. In each pod created by Kubernetes there is a container with the app. By applying a sidecar method it is possible to deploy Zeek, Suricata, Filebeat and the Wazuh agent on the same pod as individual containers. This, paired with a shared volume which aggregates specified data from the different containers in one volume, allows for full monitoring of the app processes, network interactions and system metrics.

Alongside the application stack, a separate system was configured with an installation of Wazuh Manager that would work as the communication endpoint for monitoring software, as shown in figure 4.1.

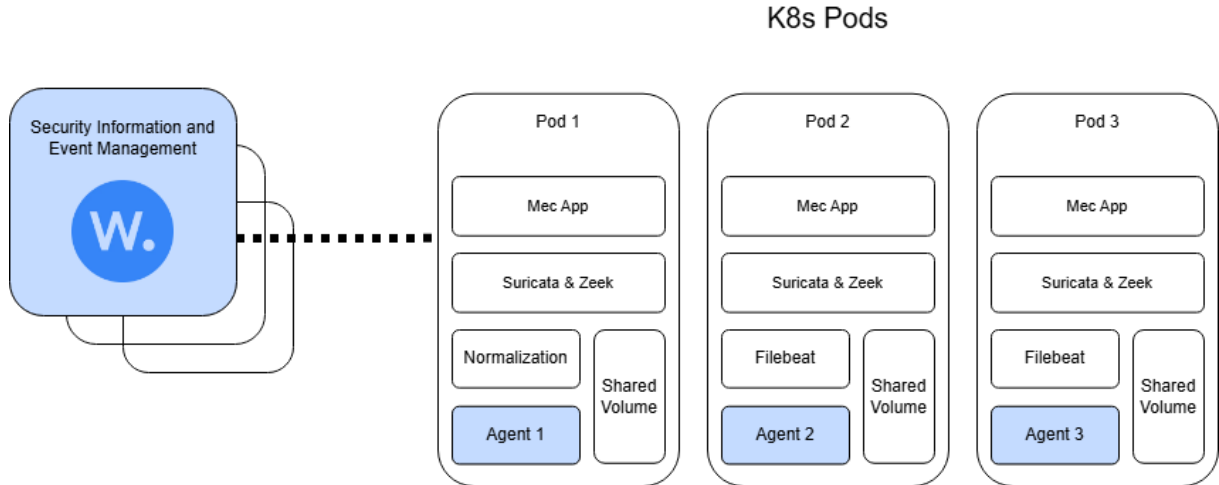


Figure 4.1: Deployment diagram of the SIEM and the monitored MEC application pods

The following figure represents the whole architecture of this project. It can be divided into two big groups, which are: developed work (MEC demo App, app deployment structure containing the monitoring software, two notification webhooks, one IP block webhook, custom App and Zeek rules, Prometheus-Wazuh compatibility bridge, VXLANs and the Firewall), configuration work (Falco, Prometheus, Suricata, Zeek, Filebeat, Wazuh Agent and the SIEM). It is relevant to note that some developed work like webhooks or compatibility scripts do not appear in the following architecture, but will be referenced between explaining each block.

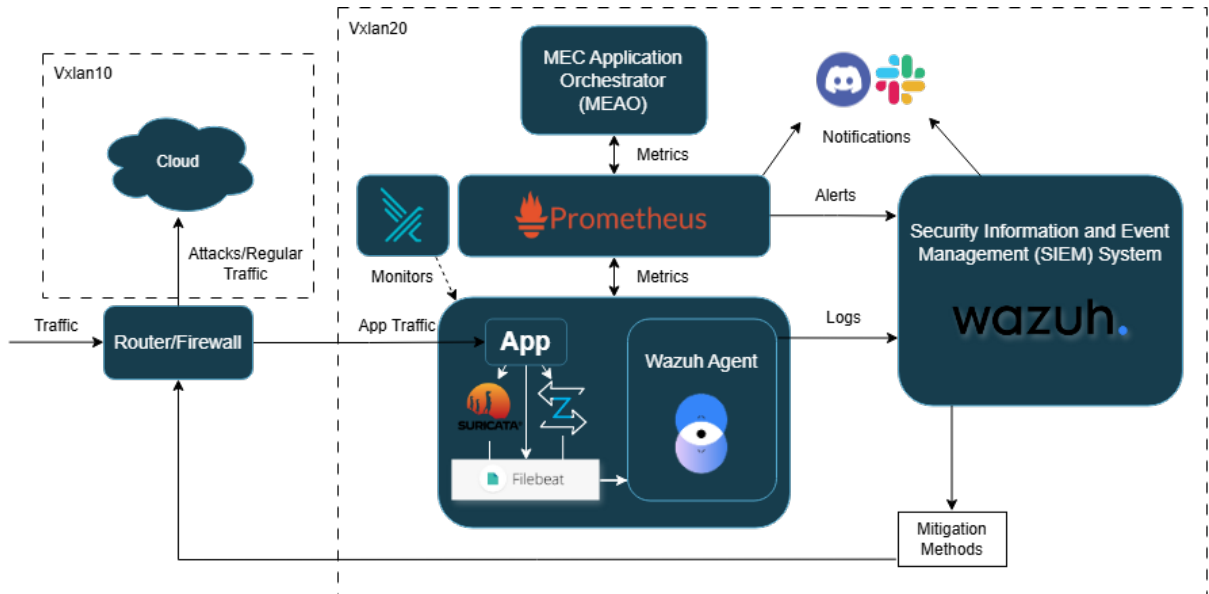


Figure 4.2: Main architecture of the project

In order to achieve a well balanced, secure and highly scalable architecture, one of the first decisions took was the use of individual images of Zeek, Suricata, Filebeat and the Wazuh Agent in each MEC App. Using this method we allow the analyst to separate each app in the Wazuh dashboard (SIEM), and allow a future implementation where if one service is under attack,

if quarantine methods are applied, the other systems will still work with no problem. This implementation also supports a better debugging experience, and allows for individual rules being applied to each app depending on the agent being used. As mentioned in the Future Work section, further testing will allow to understand if the performance impact is noticeable or if there are any other alternatives to deploy even lighter images for individual use, given that the ones in use are configured to have less impact than the standard ones.

4.2 Log Collection

In the current architecture there are 5 different sources of logs, those being:

4.2.1 Suricata, Zeek and the App

Analyse and generate logs according to the app's network interface in use. The logs go through Filebeat for a normalization and aggregation in a single file process where abnormal additions to default formats like double encapsulation in JSON logs are removed. After being normalized and converted to JSON only, the logs are collected by the corresponding Wazuh Agent and transported to the SIEM.

4.2.2 Prometheus

Scrapes metrics by making requests to Node Exporter instances across all systems, which provide telemetry such as CPU, memory, and network usage, as well as from cAdvisor, which exposes per-container resource consumption. These metrics are continuously evaluated against a set of custom alerting rules. When a threshold is hit for a set amount of time, the log containing the alert is sent to the module Alertmanager inside the server whose function is to dispatch the alert to a custom Prometheus-Wazuh bridge. The bridge contains a Wazuh Agent which redirects the log to the SIEM.

4.2.3 Falco

The analysis is applied to the whole pod manager, so the agent who retrieves the logs is the one located on the same machine as the Kubernetes. The logs are then created by Falco based on a deep container analysis, detecting abnormal behaviours like privilege escalations and file tampering, added to a file shared with agent, collected and transported to the SIEM.

Each Wazuh Agent connection to the SIEM is encrypted by default using the Advanced Encryption Standard (AES).

All deployed applications are expected to follow a strict log structure, so as to ensure ingestion and filtering are consistent. They may, however, extend the structure with additional fields via the `extra` array:

```

{
  timestamp: 2026-05-04T15:16:14+0000,
  application: "video-processing-app",
  alert: "INFO",
  message: "accepted connection from 10.42.0.225:5000",
  extra: []
}

```

Figure 4.3: Minimum required app logs

4.3 MEC Demo App

The main purpose of the app was to simulate a video traffic service that would be always opened. This proved not to be very important as the main goal of this project shifted to a more open concept where all the data coming in and out of the MEC App would be analysed. Despite this, the app was a crucial step to learn more about the deployment of apps in a MEC environment and the basics of container based architectures.

```

spec:
  containers:
    # APP
    - name: {{ .Values.name }}
      image: {{ .Values.deployment.image.repository }}
      imagePullPolicy: {{ .Values.deployment.image.pullPolicy }}
      volumeMounts:
        - name: shared-logs
          mountPath: /var/log/my-app
      ports:
        - containerPort: {{ .Values.deployment.containerPort }}

```

Figure 4.4: Small segment of the deployment

4.4 Network

Our initial objective regarding the network was to implement mechanisms that would rigorously differentiate and isolate the Public Outbound Traffic (e.g. visitors) from Critical Edge Traffic (e.g. security cameras). This is a critical step to ensure the massive volume of traffic coming from the Public Outbound Traffic network doesn't compromise the sensitive traffic and data present in the Critical Edge Traffic network as well as preventing Lateral Movement Attacks.

We started by evaluating the most widely used solution: 802.1Q VLANs. This mechanism is supported by many Layer 2 devices (Ethernet switches and Wi-Fi access points), and is completely invisible to network hosts. However, due to system limitations of our Proxmox instance, we resorted to VXLANs.

VXLANs work in Layer 3 by encapsulating Layer 2 MAC frames within Layer 3 UDP packets allowing us to implement secure network segments without having to rely on traditional VLAN tagging. To achieve full network isolation, a firewall was implemented to drop packets marked with a source IP that does not match the interface they arrived on. This is enforced through strict

Interface Binding combined with the Linux kernel's built-in Unicast Reverse Path Forwarding (uRPF), adhering to the anti-spoofing principles of BCP 38. As a result, even if an attacker perfectly spoofs an internal trusted IP address, any packet arriving on the wrong physical cable will be silently dropped.

One key feature we have in-place is the automated blocking of IPs behaving in a suspicious manner. This feature works by having a service listening on a Webhook, and then proceeds to block it through UFW while at the same time shutting off the connection of the affected IP, this is due to a quirk of the firewall being stateful. We also had to take in mind possible spoofing attacks or attacks similar to an SQL injection where attackers parse unwanted commands through the Webhook. To prevent spoofing attacks and blocking of unwanted IPs we created a whitelist containing the IPs of the machines handling Critical Edge Traffic, on the other hand a small valid IP check was implemented to prevent injection attacks.

4.5 Rules

Custom rules configured in the Wazuh Manager (SIEM) enable pattern identification of threats, followed by alerts to allow the analyst to inspect and act on flagged events immediately. Regarding the logs coming from the app, the rules which inspect them can easily be changed to support any kind of information and output.

```
<rule id="100011" level="0">
  <if_sid>100010</if_sid>
  <field name="level">^DEBUG$</field>
  <description>Application DEBUG log.</description>
</rule>

<rule id="100012" level="3">
  <if_sid>100010</if_sid>
  <field name="level">^INFO$</field>
  <description>Application INFO log: $(message)</description>
</rule>

<rule id="100013" level="5">
  <if_sid>100010</if_sid>
  <field name="level">^ERROR$</field>
  <description>Application ERROR log: $(message)</description>
</rule>
```

Figure 4.5: App demonstrative rules

The rules consist of a XML file that analyses selected logs with tags and allow for pattern identification. For example, when Zeek detects a connection to an endpoint that contains an expired certificate, it can generate an alert for the analyst to see, which contains a message saying from where to where it happened, as seen in figure 4.6.

```
<rule id="100907" level="12">
  <if_sid>100900</if_sid>
  <field name="validation_status">certificate has expired</field>
  <description>Zeek: Client $(zeek_id.orig_h) connected to a server with expired certificate $(zeek_id.resp_h)</description>
</rule>
```

Figure 4.6: Rule that identifies the expired certificate tag in the log

Using the rules it is also possible to identify more complex patterns like how many logs of one type occur during a time interval. A good example of this is the identification of a port scanning event. Using this technique the SIEM can look for rejected packages of TCP connections, if it detects a large amount of these rejected tries in a small amount of time, it alerts the analyst for a possible port scanning event.

```
<rule id="100903" level="7">
  <if_sid>100900</if_sid>
  <field name="conn_state">REJ</field>
  <description>Zeek: Connection from $(zeek_id.orig_h) to $(zeek_id.resp_h) $(zeek_id.resp_p) rejected</description>
</rule>

<rule id="100904" level="10" frequency="5" timeframe="20">
  <if_matched_sid>100903</if_matched_sid>
  <same_field>zeek_id.orig_h</same_field>
  <description>Zeek: Multiple rejected connections from $(zeek_id.orig_h) (5+ in 20s - possible port scan activity)</description>
</rule>
```

Figure 4.7: Rule that identify port scanning

4.6 Whitelisted IPs

To complement the whitelist actions taken from the firewall, the monitoring system also alerts for possible non-whitelisted connections using the logs from Zeek. It starts by reading a file that contains a list of whitelisted IPs previously added (e.g. venue computers that require access to certain MEC services) like the one in figure 4.8. It then compares the ones on the list with the source IP from the connection. If the IP does not match it alerts the analyst.

```
192.168.1.10:camera_1
192.168.1.11:machine_A
```

Figure 4.8: Format of whitelisted IPs

```
<rule id="100908" level="10">
  <if_sid>100900</if_sid>
  <field name="zeek_id.orig_h">\.+.</field>
  <list field="zeek_id.orig_h" lookup="not_address_match_key">etc/lists/whitelist-mec-apps</list>
  <description>Security Alert: Connection not from Whitelisted IP $(zeek_id.orig_h) $(zeek_id.resp_h)</description>
</rule>
```

Figure 4.9: Rule that alerts the user of a non-whitelisted connection

Although this system is very rudimentary, its only goal is to demonstrate the integration of a checklist method applied alongside the rule set.

4.7 Metrics-based Alerts

Prometheus automatically runs its metrics against a set of custom hot-reloadable rules every 15 seconds and, upon detecting anomalous behaviour, triggers Alertmanager to dispatch notifications to both Discord and to a lightweight Python webhook service. The service normalizes each incoming alert and forwards it to Wazuh, where it can be processed and displayed alongside the rest of the security alerts.

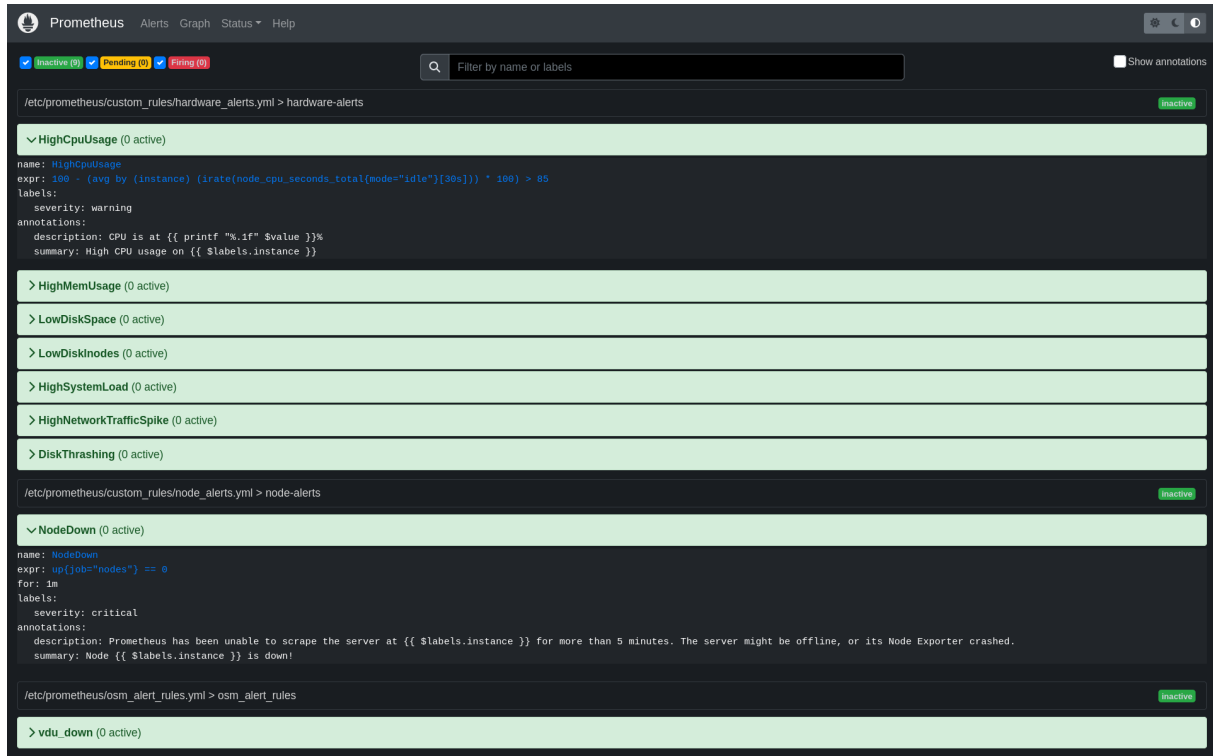


Figure 4.10: List of hardware alert rules inside Prometheus

4.8 Log History

All the security events above level 3 of importance are archived in Wazuh by default for a minimum of 30 days. This makes it possible to access a history of past events, which could help in case of forensic analysis. As mentioned in the Future Work section this would be a case of study to optimize the amount of logs stored by only leaving important events and contextual logs.

Chapter 5

Results

5.1 Security Information and Event Management SIEM Integration

A fully operational SIEM system based on Wazuh was deployed and integrated into the MEC infrastructure. The Wazuh dashboard, shown in Figure 5.1, provides a centralized overview of the security posture across all agents, with alert counts broken down by severity level and access to modules such as Threat Hunting, MITRE ATT&CK mapping, and File Integrity Monitoring.

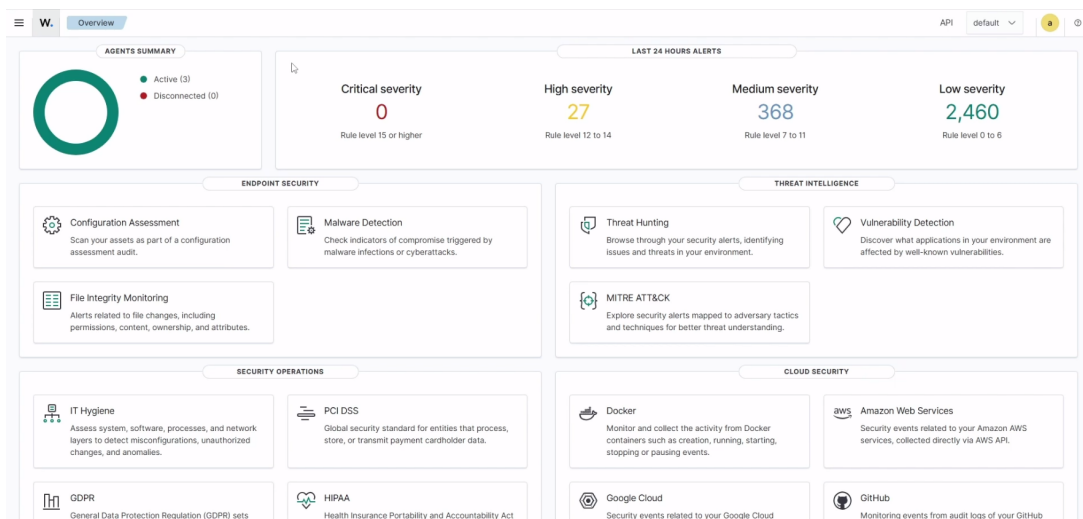


Figure 5.1: Wazuh Dashboard displaying alert severity breakdown and available security modules

Three dedicated agents were registered and maintained active throughout the deployment: the MEC host agent, the video processing application agent, and the Alertmanager-Wazuh bridge agent, as shown in Figure 5.2. This per-agent isolation ensures that telemetry and log data remain separated at the application level, so that a compromise in one pod does not propagate visibility to others.

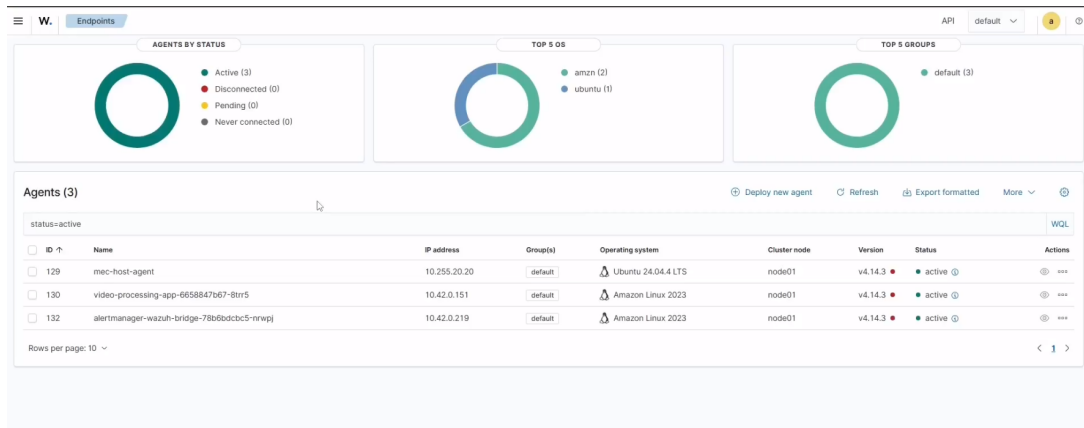


Figure 5.2: Three deployed and active agents registered in the MEC deployment

Analysts can inspect each individual agents to see the full event stream. Figure 5.3 shows the Threat Hunting view filtered to the video processing application agent, where logs from Suricata and application-level rules are displayed side by side, enabling rapid triage of flagged events.

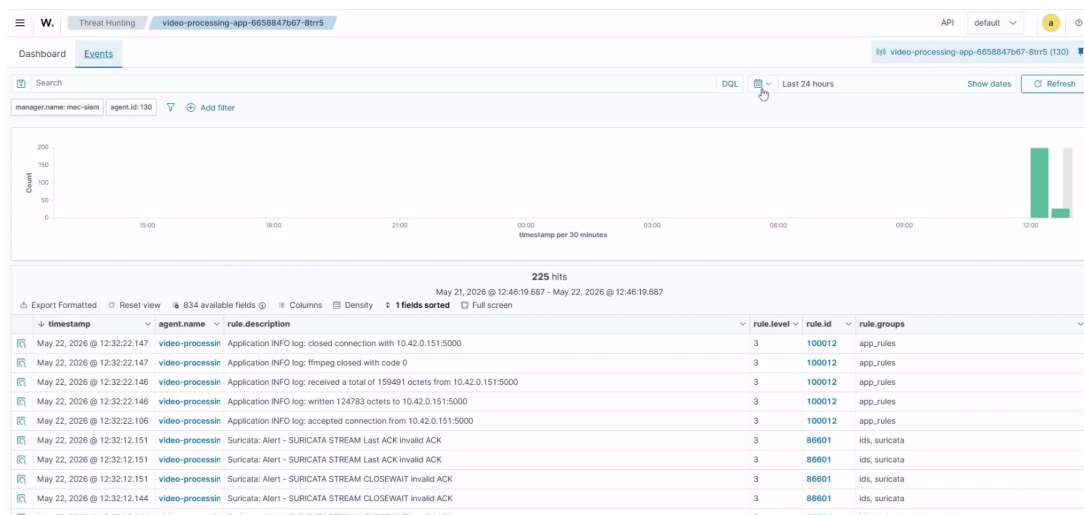


Figure 5.3: Dashboard showing the video processing application agent, with Suricata and application log events

5.2 Monitoring and Telemetry

Infrastructure health metrics are collected by Prometheus and visualized through two Grafana dashboards. The per-node dashboard, shown in Figure 5.4, exposes host-level telemetry including CPU utilisation, memory pressure, and network throughput for each physical node in the deployment. The Kubernetes cluster dashboard, shown in Figure 5.5, provides an aggregate view of all pods, covering memory and CPU ratios, namespace resource statistics, and real-time network throughput across the cluster.

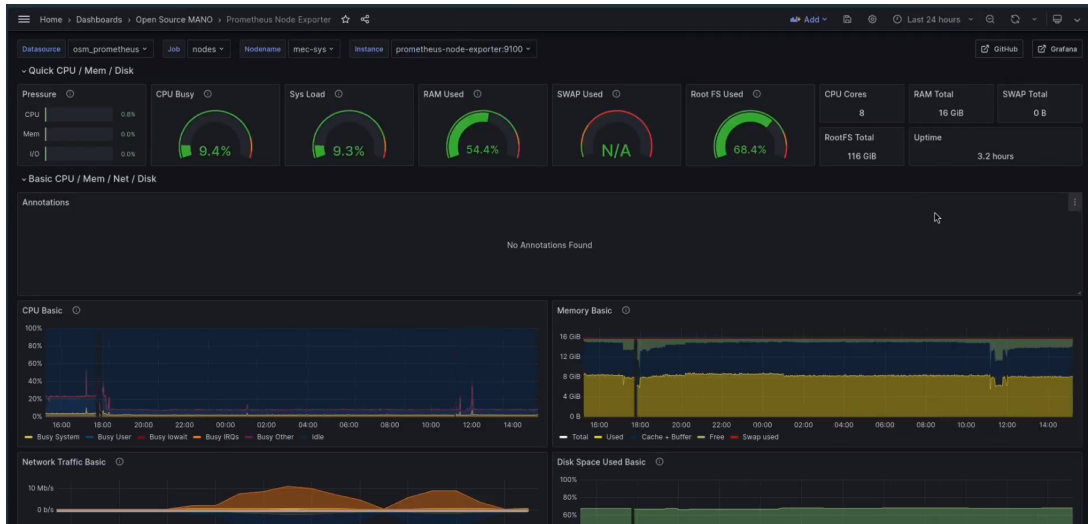


Figure 5.4: Grafana dashboard showing per-node CPU, memory, and network telemetry collected via Prometheus Node Exporter.



Figure 5.5: Grafana Kubernetes cluster dashboard displaying aggregate resource usage across all nodes and pods.

When a metric crosses a configured threshold, Alertmanager dispatches a notification to Discord and forwards the alert to the Wazuh manager via the Prometheus-Wazuh bridge, keeping infrastructure and security events unified in a single platform.

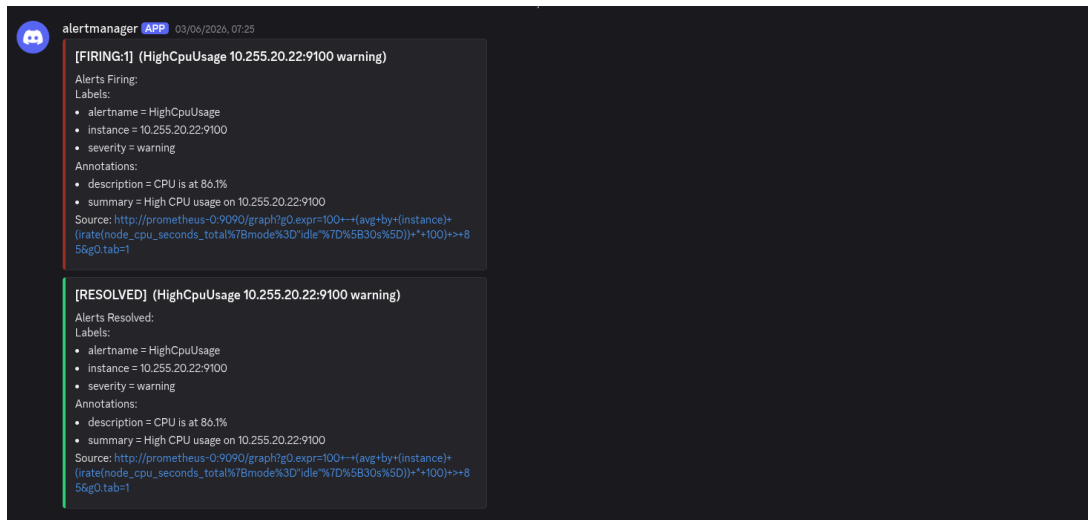


Figure 5.6: Example of a Discord notification dispatched by Alertmanager upon detection of high CPU usage (top) and the corresponding resolution notification once the metric returned to normal (bottom)

5.3 Threat Detection and Automated Response

Although not tested to its full potential, the current implementation detects various threats such as ssh brute-force, unauthorized lateral movement between pods, resource exhaustion conditions, port scanning, invalid or inexistent certificates, privilege escalation, file tampering, and many other custom and default rules/identifiers.

To amplify the importance of some events, any log greater or equal to level 10 is sent through a webhook to Discord. It relevant to note that any other system or communication platform that allows webhooks could be used (e.g. Slack).

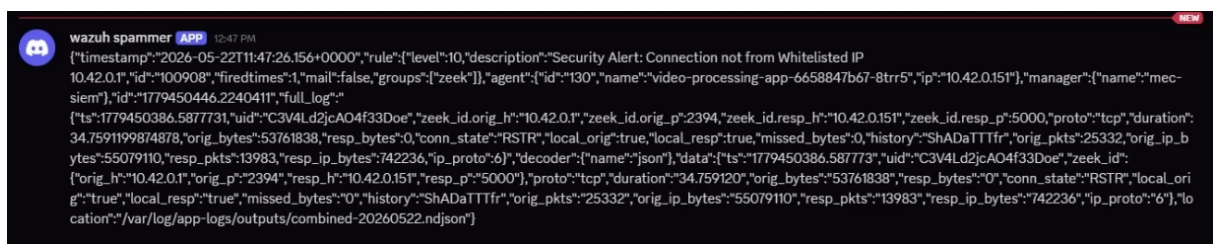


Figure 5.7: Example of a Discord notification triggered by a Zeek alert for a connection originating from a non-whitelisted IP address.

The automated response mechanism was validated through a simulated SSH brute-force attack, shown in Figure 5.8. Repeated failed login attempts from an external host were detected by the SIEM, which creates a set of alerts in the Wazuh Dashboard. If the brute-force attack is successful, it triggers a webhook that informs the Firewall which IP to block. The webhook ban log is visible in the figure 5.9 and confirms that the IP was successfully blocked.

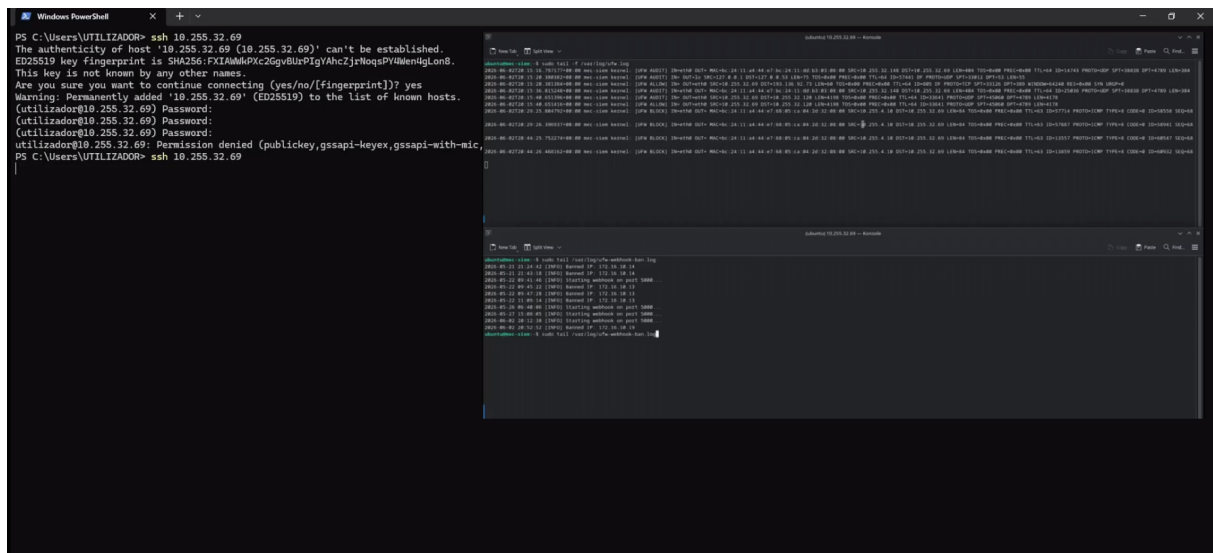


Figure 5.8: Simulated SSH brute-force attack from an external host (left), UFW block logs at the firewall (top right), and the automated IP ban webhook log confirming mitigation (bottom right).

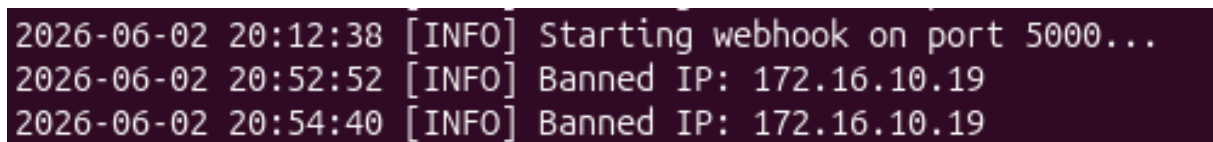


Figure 5.9: Logs from the firewall

5.4 Network Isolation and Traffic Segmentation

Strict separation between Public Outbound Traffic and Critical Edge Traffic was achieved through a VXLAN-based segmentation scheme. The effectiveness of this isolation was verified empirically through two complementary tests shown in Figures 5.10 and 5.11.

Figure 5.10 demonstrates the cross-segment blocking behaviour: a host on Vxlan10 attempting to reach a host on Vxlan20 observes 100% packet loss, while communication within the same segment succeeds. The UFW block entries visible in the lower terminal confirm that packets from the external segment are silently dropped by the firewall before reaching any internal MEC application pod.

```

ubuntu10.255.32.69 ~$ ping 10.255.32.62
PING 10.255.32.62 (10.255.32.62) 56(84) bytes of data:
^C
--- 10.255.32.62 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2806ms

atnuguser10.255.32.98 ~$ ping 10.255.32.62
PING 10.255.32.62 (10.255.32.62) 56(84) bytes of data:
^C
--- 10.255.32.62 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2047ms

atnuguser10.255.32.98 ~$ ping 10.255.10.10
PING 10.255.10.10 (10.255.10.10) 56(84) bytes of data:
64 bytes from 10.255.10.10: icmp_seq=1 ttl=64 time=0.385 ms
64 bytes from 10.255.10.10: icmp_seq=2 ttl=64 time=0.324 ms
64 bytes from 10.255.10.10: icmp_seq=3 ttl=64 time=0.208 ms
64 bytes from 10.255.10.10: icmp_seq=4 ttl=64 time=0.341 ms
64 bytes from 10.255.10.10: icmp_seq=5 ttl=64 time=0.338 ms
^C
--- 10.255.10.10 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4180ms
rtt min/avg/max/mdev = 0.208/0.319/0.341/0.010 ms

2026-05-22T11:30:17.876666+00:00 test-user1 kernel: [UFW AUDIT] IN=eth0 OUT= MAC=bc:24:11:f0:5a:4d:bc:24:11:a4:44:e7:08:00 SRC=10.255.32.69 DST=10.255.32.62 LEN=64 TOS=0x00 PREC=0x00 TTL=64 ID=61228 DF PROTO=ICMP TYPE=8 CODE=0 ID=37878 SEQ=2
2026-05-22T11:30:17.876687+00:00 test-user1 kernel: [UFW BLOCK] IN=eth0 OUT= MAC=bc:24:11:f0:5a:4d:bc:24:11:a4:44:e7:08:00 SRC=10.255.32.69 DST=10.255.32.62 LEN=64 TOS=0x00 PREC=0x00 TTL=64 ID=61228 DF PROTO=ICMP TYPE=8 CODE=0 ID=37878 SEQ=2
2026-05-22T11:30:18.900681+00:00 test-user1 kernel: [UFW AUDIT] IN=eth0 OUT= MAC=bc:24:11:f0:5a:4d:bc:24:11:a4:44:e7:08:00 SRC=10.255.32.69 DST=10.255.32.62 LEN=64 TOS=0x00 PREC=0x00 TTL=64 ID=61499 DF PROTO=ICMP TYPE=8 CODE=0 ID=37878 SEQ=3
2026-05-22T11:30:18.900706+00:00 test-user1 kernel: [UFW BLOCK] IN=eth0 OUT= MAC=bc:24:11:f0:5a:4d:bc:24:11:a4:44:e7:08:00 SRC=10.255.32.69 DST=10.255.32.62 LEN=64 TOS=0x00 PREC=0x00 TTL=64 ID=61499 DF PROTO=ICMP TYPE=8 CODE=0 ID=37878 SEQ=3
2026-05-22T11:30:27.047799+00:00 test-user1 kernel: [UFW BLOCK] IN=eth0 OUT= MAC=bc:24:11:f0:5a:4d:bc:24:11:0f:92:e4:08:00 SRC=10.255.32.98 DST=10.255.32.62 LEN=64 TOS=0x00 PREC=0x00 TTL=64 ID=21412 DF PROTO=ICMP TYPE=8 CODE=0 ID=55987 SEQ=1
2026-05-22T11:30:28.071811+00:00 test-user1 kernel: [UFW BLOCK] IN=eth0 OUT= MAC=bc:24:11:f0:5a:4d:bc:24:11:0f:92:e4:08:00 SRC=10.255.32.98 DST=10.255.32.62 LEN=64 TOS=0x00 PREC=0x00 TTL=64 ID=21508 DF PROTO=ICMP TYPE=8 CODE=0 ID=55987 SEQ=2
2026-05-22T11:30:29.095857+00:00 test-user1 kernel: [UFW BLOCK] IN=eth0 OUT= MAC=bc:24:11:f0:5a:4d:bc:24:11:0f:92:e4:08:00 SRC=10.255.32.98 DST=10.255.32.62 LEN=64 TOS=0x00 PREC=0x00 TTL=64 ID=22599 DF PROTO=ICMP TYPE=8 CODE=0 ID=55987 SEQ=3
2026-05-22T11:30:32.866892+00:00 test-user1 kernel: [UFW AUDIT] IN=eth0 OUT= MAC=bc:24:11:f0:5a:4d:bc:24:11:0f:92:e4:08:00 SRC=10.255.32.98 DST=10.255.32.62 LEN=134 TOS=0x00 PREC=0x00 TTL=64 ID=28061 PROTO=UDP SPT=51389 DPT=4789 LEN=114
2026-05-22T11:30:32.866923+00:00 test-user1 kernel: [UFW ALLOW] IN= OUT=eth0 SRC=10.255.32.62 DST=10.255.32.98 LEN=134 TOS=0x00 PREC=0x00 TTL=64 ID=38081 PROTO=UDP SPT=54528 DPT=4789 LEN=114
2026-05-22T11:30:33.899313+00:00 test-user1 kernel: [UFW AUDIT] IN= OUT=eth0 SRC=10.255.32.62 DST=10.255.32.98 LEN=134 TOS=0x00 PREC=0x00 TTL=64 ID=33221 PROTO=UDP SPT=54528 DPT=4789 LEN=114

```

Figure 5.10: Empirical verification of VXLAN-based network isolation: cross-segment ping attempts result in 100% packet loss, with UFW block entries confirming firewall enforcement.

Figure 5.11 shows throughput measurements using `iperf3` within each segment. The MEC internal segment sustains approximately 4 Gbits/s, while the public segment achieves around 1.91 Gbits/s, confirming that both networks operate independently at full capacity without mutual interference.

```

Server listening on 5201 (test #1)
Accepted connection from 10.255.20.20, port 68902
[ 5 ] local 10.255.20.22 port 5201 connected to 10.255.20.20 port 68904
[ ID ] Interval      Transfer      Bitrate
[ 5 ] 0.00-1.00 sec  483 MBytes  4.05 Gbits/sec
[ 5 ] 1.00-2.00 sec  498 MBytes  4.18 Gbits/sec
[ 5 ] 2.00-3.00 sec  482 MBytes  4.04 Gbits/sec
[ 5 ] 3.00-4.00 sec  510 MBytes  4.28 Gbits/sec
[ 5 ] 4.00-5.00 sec  562 MBytes  4.72 Gbits/sec
[ 5 ] 5.00-6.00 sec  442 MBytes  3.71 Gbits/sec
[ 5 ] 6.00-7.00 sec  425 MBytes  3.57 Gbits/sec
[ 5 ] 7.00-8.00 sec  433 MBytes  3.64 Gbits/sec
[ 5 ] 8.00-9.00 sec  580 MBytes  4.86 Gbits/sec
[ 5 ] 9.00-10.00 sec  472 MBytes  3.96 Gbits/sec
[ 5 ] 10.00-10.00 sec  768 Kbytes  4.33 Gbits/sec
[ ID ] Interval      Transfer      Bitrate
[ 5 ] 0.00-10.00 sec  4.70 Gbytes  4.04 Gbits/sec
Server listening on 5201 (test #2)

atnuguser10.255.32.148 ~$ iperf3 -c 10.255.20.22
Connecting to host 10.255.20.22, port 5201
[ 5 ] local 10.255.20.20 port 68904 connected to 10.255.20.22 port 5201
[ ID ] Interval      Transfer      Bitrate      Retx      Cwnd
[ 5 ] 0.00-1.00 sec  486 MBytes  4.07 Gbits/sec  0  1.65 Mbytes
[ 5 ] 1.00-2.00 sec  499 MBytes  4.19 Gbits/sec  0  1.84 Mbytes
[ 5 ] 2.00-3.00 sec  481 MBytes  4.03 Gbits/sec  0  1.93 Mbytes
[ 5 ] 3.00-4.00 sec  580 MBytes  4.87 Gbits/sec  0  1.93 Mbytes
[ 5 ] 4.00-5.00 sec  562 MBytes  4.71 Gbits/sec  0  2.03 Mbytes
[ 5 ] 5.00-6.00 sec  441 MBytes  3.70 Gbits/sec  0  2.03 Mbytes
[ 5 ] 6.00-7.00 sec  425 MBytes  3.57 Gbits/sec  0  2.03 Mbytes
[ 5 ] 7.00-8.00 sec  433 MBytes  3.63 Gbits/sec  0  2.03 Mbytes
[ 5 ] 8.00-9.00 sec  580 MBytes  4.87 Gbits/sec  0  2.14 Mbytes
[ 5 ] 9.00-10.00 sec  472 MBytes  3.95 Gbits/sec  0  2.14 Mbytes
[ ID ] Interval      Transfer      Bitrate      Retx      Cwnd
[ 5 ] 0.00-10.00 sec  4.71 Gbytes  4.04 Gbits/sec  0  sender
[ 5 ] 0.00-10.00 sec  4.70 Gbytes  4.04 Gbits/sec  0  receiver
iperf3 Done

atnuguser10.255.32.98 ~$ iperf3 -c 10.255.10.11
Connecting to host 10.255.10.11, port 5201
[ 5 ] local 10.255.10.10 port 37876 connected to 10.255.10.11 port 5201
[ ID ] Interval      Transfer      Bitrate      Retx      Cwnd
[ 5 ] 0.00-1.00 sec  221 MBytes  1.91 Gbits/sec  0  884 Kbytes
[ 5 ] 1.00-2.00 sec  228 MBytes  1.91 Gbits/sec  0  884 Kbytes
[ 5 ] 2.00-3.00 sec  228 MBytes  1.91 Gbits/sec  0  884 Kbytes
[ 5 ] 3.00-4.00 sec  228 MBytes  1.92 Gbits/sec  0  842 Kbytes
[ 5 ] 4.00-5.00 sec  227 MBytes  1.90 Gbits/sec  0  842 Kbytes
[ 5 ] 5.00-6.00 sec  227 MBytes  1.91 Gbits/sec  0  842 Kbytes
[ 5 ] 6.00-7.00 sec  228 MBytes  1.92 Gbits/sec  0  893 Kbytes
[ 5 ] 7.00-8.00 sec  227 MBytes  1.90 Gbits/sec  0  893 Kbytes
[ 5 ] 8.00-9.00 sec  228 MBytes  1.91 Gbits/sec  0  893 Kbytes
[ 5 ] 9.00-10.00 sec  228 MBytes  1.91 Gbits/sec  0  900 Kbytes
[ ID ] Interval      Transfer      Bitrate      Retx      Cwnd
[ 5 ] 0.00-10.00 sec  2.23 Gbytes  1.91 Gbits/sec  0  sender
[ 5 ] 0.00-10.00 sec  2.22 Gbytes  1.91 Gbits/sec  0  receiver
iperf3 Done

```

Figure 5.11: Throughput measurements using `iperf3` within the MEC internal segment (left, ~4 Gbits/s) and the public segment (right, ~1.91 Gbits/s), confirming independent operation of both networks.

Chapter 6

Conclusion

To conclude, this project took a very exploratory approach to improving security and operational visibility in high density and traffic intensive scenarios, where services are deployed as k8s pods. Despite the many constraints, the result is a well-balanced and versatile framework for use in any k8s-based edge deployment.

Although the monitoring system is just one small component in a broader security plan, this project emphasizes the importance of centralizing network visibility and awareness, as fragmented tools can significantly slow response times down and increase the chance of missed events.

Alongside all initial goals being met, several additional features were also added during development.

6.1 Future work

As mentioned, this work inserts into a larger environment with preventive and mitigation responses. Among future objectives would be the implementation of automated log cross analysis joining the values coming from the metrics and the system logs, for example automatically cross-referencing a network-level alert with real time container anomalies (e.g. spike in CPU or memory usage).

One major future goal would be the integration of more MEC specific functions, as this project ended up in a more conventional monitoring architecture. Unfortunately this was due to a lack of knowledge base about these systems and what possibilities do they offer in terms of control over information.

The current automated mitigation mechanisms could also be expanded to include service isolation, localized rate-limiting, or automated pod recycling when an application is compromised.

Dynamic Quarantine Networks are an interesting option to look into, expanding and building upon the current VXLAN architecture by dynamically setting up secure, isolated VLANs/VXLANs. These network segments would work as isolated live monitoring environments where compromised application pod traffic or malicious traffic could be rerouted to for analysis and containment.

Although in theory the current framework was done in order to sustain high-density traffic, it would be important to apply more stress tests and simulate possible attacks coming from both inside and outside the network. This would allow to find possible breaking points and bottlenecks in the current architecture, and reinforce the security in the monitoring system.

One other future addition could be the automation of reports with context, which in the event of a flagged security breach, would automatically aggregate and archive a timeline of relevant logs, network captures and system metrics, outputting a comprehensive compliance and forensic report.

Bibliography

- [1] ETSI. *Mobile Edge Computing (MEC); Mobile Edge Management; Part 2: Application life-cycle, rules and requirements management*. Tech. rep. Feb. 2024. URL: https://www.etsi.org/deliver/etsi_gs/MEC/001_099/01002/02.02.01_60/gs_MEC01002v020201p.pdf (visited on 03/20/2026).
- [2] Pedro Ferreira et al. *MEC in OSM*. Version 1.1. Sept. 2024. DOI: 10.5281/zenodo.13862189. URL: <https://github.com/ATNoG/osm-mec/tree/ieee-nfv-sdn-paper-v1.1>.
- [3] Pedro Pereira et al. “Automatic Service Migration in a MEC-NFV Environment”. In: *Proc IEEE Conference on Network Function Virtualization and Software Defined Networks NFV-SDN*. Natal, Brazil, Nov. 2024. DOI: 10.1109/NFV-SDN61811.2024.10807471.